

增強課程 3：Classic BAdI 觀念與尋找

Lecture

en02 已經實際用過 Classic User-Exit (`CALL CUSTOMER-FUNCTION`)。這題要講的 **BAdI (Business Add-In)** 是 User-Exit 的物件導向化後繼者 (NetWeaver 4.6 起)——概念上做同一件事 (讓 SAP 標準流程留一個「呼叫外掛」的插入點)，但呼叫機制從「呼叫一個寫死的 Function Module 編號」換成「呼叫一個 Interface 的方法」，因此天生擁有物件導向的好處：可以有多个實作並存、可以用 Filter 依條件分流、實作時能繼承共用邏輯。

兩個角色，兩個交易碼：

- **Definition (SE18)**：定義「這裡有一個可以被擴充的點」——本質上是宣告一個 **Interface** (命名慣例 `IF_EX_<BAdI 名稱>`)，這個 Interface 必須繼承 `IF_BADI_INTERFACE` (一個沒有任何方法的「標記介面」，純粹讓 Kernel 認得這是一個合法的 BAdI Interface，缺了這個繼承，這個 Interface 就只是普通 Interface，不能拿來做 BAdI)。Definition 還決定這個 BAdI 是 **Single Use** (全系統只能有一個 Implementation，en02 用過的 `BADI_BATCH_NUMBER_INT` 就是這種) 還是 **Multi Use** (可以有多个 Implementation 並存，en01 用過的 `MB_MIGO_BADI` 是這種)。
- **Implementation (SE19)**：寫一個 **Class** 實作這個 Interface，把真正的客製化邏輯填進方法本體。Multi Use 的 BAdI 可以有很多個 Implementation 同時存在。

Filter-dependent (篩選相依)：Multi Use 的 BAdI 除了讓多个 Implementation 並存，還可以宣告一個 **Filter** (例如公司代碼、工廠、業務類型)，執行時系統依當下的 Filter 值只呼叫符合條件的 Implementation——這讓「不同事業體用不同客製化邏輯」不需要在程式碼裡寫一堆 `IF` 判斷，而是宣告式地讓框架自動分流。

呼叫語法 `GET BADI / CALL BADI` (en02 讀 `VB_NEXT_BATCH_NUMBER` 時已經看過真實用法)：

```
DATA go_badi TYPE REF TO if_ex_badi_mm_matnr.
TRY.
    GET BADI go_badi.
    CALL BADI go_badi->check_mara
        EXPORTING
            mdata = ls_mara
        EXCEPTIONS
            in_use = 1.
CATCH cx_badi_not_implemented.
    " Single Use 且沒有 Implementation 時會走到這裡；
    " Multi Use 即使 0 個 Implementation，GET BADI / CALL BADI 通常正常執行、
    " 只是不會呼叫到任何實際邏輯，兩者的「沒人實作」表現方式不同。
ENDTRY.
```

⚠ 這組新式關鍵字語法不是每個 BAdI 都能用——實測發現：`GET BADI/CALL BADI` 只適用於有 `ENHS/XS` (**Enhancement Spot**) 身分的 BAdI (en01 的 `MB_MIGO_BADI`、本題的 `BADI_MM_MATNR` 都同時有 `SXSD/XD` [舊] + `ENHS/XS` [新，NW 7.0 統一框架時自動包上的外殼])；但有些 Classic BAdI 只有 `SXSD/XD`，完全沒有 `ENHS/XS` (例如 PP / `COOIS` 用的 `WORKORDER_INFOSYSTEM`)，這種「純舊式」BAdI 用 `GET BADI lo_badi`，直接在編譯階段就報錯「"LO_BADI" is not a valid BAdI handle here」，必須改用更早期的呼叫慣例：

```
DATA go_exit TYPE REF TO if_ex_workorder_infosystem.
CALL METHOD cl_exithandler=>get_instance
    EXPORTING exit_name = 'WORKORDER_INFOSYSTEM'
    CHANGING instance = go_exit
    EXCEPTIONS no_reference = 1 OTHERS = 2.
IF go_exit IS BOUND.
    CALL METHOD go_exit->method_name ...
ENDIF.
```

`cl_exithandler=>get_instance` 對 Multi Use BAdI 一律成功回傳一個 **bound 的 instance** (它本質上是個「分派器」，內部管理 0~多个 Implementation)，即使 0 個生效中的實作也不會報錯，呼叫方法就是安靜地什麼都不做——這點跟新式語法對 Single Use、沒實作時丟 `CX_BADI_NOT_IMPLEMENTED` 是不同的「沒實作」表現方式，但殊途同歸：呼叫端都不需要特別處理「完全沒人實作」這種狀況會不會讓程式掛掉。判斷該用哪種語法：quickSearch 該 BAdI 名稱，看有沒有 `ENHS/XS` 型別的結果，沒有就要用 `cl_exithandler=>get_instance`。

Implementation「存在」不等於「生效」，這裡有獨立的 **Active 開關** (跟 en02 的 CMOD Project Activate 概念類似，但更輕量)：本題查證到一個真實案例——SAP 標準 BAdI `BADI_MM_MATNR` (Definition 說明「Modification-Free Archiving Enhancement of MM_MATNR」，套件 `MGA`，Multi Use，Interface `IF_EX_BADI_MM_MATNR`) 底下有一個 Implementation `BADI_MM_MATNR` (Implementing Class `CL_IM_BADI_MM_MATNR`，套件 `CINBADI`，說明「Implementation for India」) 的 `isActive="false"`，查詢結果附帶一句直白的說明：“The implementation will not be called”——這證實一個 BAdI Implementation 可以完整寫好、存在系統裡，但只要沒被 Active，執行時就是完全被跳過。⚠ 但這只是 6 個 Implementation 裡的 1 個——第一版教材只查到這一個就下結論「這個 BAdI 沒有生效中的實作」，是誤判：改用 quickSearch 撈同名前綴 + `objectType=ENHO/XH` 篩選，發現這個 Multi Use BAdI 其實同時有 6 個 Implementation，其中 4 個是 **Active** (`EAMWS_BADI_MM_MATNR` / `CRMS4_BADI_MM_MATNR_PRDBDL_CHK` / `CRMS4_BADI_MM_MATNR_PRDBDL_ARC` / `CRMS4_BADI_MM_MATNR_ORDER_CHK`，分屬 EAM 與 CRM S/4 整合領域)，只有 2 個是 Inactive (`BADI_MM_MATNR` / `GPD_MM_MATNR_CHECK`)。實際用 `cl_exithandler=>get_instance` 呼叫 `check_mara` 方法 (帶一個空白的 `MARA` 測試資料) 時，`sy-subrc = 1` (`IN_USE` 例外被觸發)——證實真的有效中的實作在跑，不是「沒人實作」。教訓：查一個 Multi Use BAdI 的實作狀態，一定要撈全部同名前綴的 Implementation，不能只看第一筆或跟 Definition 同名的那一筆就下結論。

三個真實案例對照 (本課程實際查證過的三個標準 BAdI)：

BAdI	Single/Multi Use	Interface	呼叫語法	目前狀態
MB_MIGO_BADI (en01)	Multi Use (singleUse="false")	IF_EX_MB_MIGO_BADI	新式 (有 ENHS/XS)	尚未查過有沒有 Implementation
BADI_BATCH_NUMBER_INT (en02)	Single Use (singleUse="true")	IF_EX_BADI_BATCH_NUMBER_INT	新式 (有 ENHS/XS)	這套系統上從未被實作過
BADI_MM_MATNR (本題)	Multi Use (singleUse="false")	IF_EX_BADI_MM_MATNR	新式 (有 ENHS/XS)	6 個 Implementation · 4 個 Active
WORKORDER_INFOSYSTEM (PP / COOIS · 本題延伸實作)	Multi Use (singleUse="false")	IF_EX_WORKORDER_INFOSYSTEM	舊式 (只有 SXSD/XD · 須用 cl_exithandler=>get_instance)	至少 4 個既有 Implementation (ZWORKORDER_GOODSMVT / ZWORKORDER_INFOSYSTEM / ZWORKORDER_UPDATE / 新建的 ZWORKORDER_INFO) · 均為公司真實客製化

延伸實戰：真實新增 COOIS 自訂欄位 (WORKORDER_INFOSYSTEM) ——本題過程中順勢完成一個真實需求：在 COOIS (生產訂單資訊系統) 的表頭清單加一個「External Material Group」欄位。做法：

- CI_IOHEADER (DDIC Structure · 新增欄位 ZZEXTWG 引用 Data Element EXTWG) ——IOHEADER (COOIS 表頭清單底層結構) 本來就用新式 define structure { include ci_ioheader; ... } 語法預留了這個擴充位置 · CI_IOHEADER 本身之前不存在 (GET 是 404) · 只要照一般 DDIC Structure 建立流程把它建出來 · IOHEADER 自動吃到新欄位 · 完全不用碰 IOHEADER 本身。
- SE19 建 Implementation ZWORKORDER_INFO (GUI-only · WORKORDER_INFOSYSTEM 沒有 ENHS/XS · Claude 無法用 ADT 直接建立) · 選 TABLES_MODIFY_LAY 方法。
- Implementing Class ZCL_IM_WORKORDER_INFO (一旦 SE19 生成骨架 · 這就是普通 Class · Claude 可以用 ADT 正常讀寫) ——邏輯：迴圈 CT_IOHEADER · 依 MATNR 查 MARA-EXTWG · 寫回 ZZEXTWG。
- 重要事前檢查：動手前先查了同一個 BAdI 底下已經有一個真實同事 (MAVIS) 建立的 Implementation ZWORKORDER_INFOSYSTE (套件 ZPP · 2024 年建立 · 說明也是「COOIS 加自訂欄位」) · 讀出來是對另一張表 (CT_IOOPCOMP · 元件表) 加另一個欄位 (ZZAUART · 工單類型) · 確認兩者不衝突 (不同表、不同欄位) 才繼續——這正是 en02 學到的「動手前查證既有物件歸屬」在 BAdI 世界的體現。
- 端對端測試成功：使用者在 SE19 Activate ZWORKORDER_INFO 後 · COOIS 執行清單、Layout 加入 ZZEXTWG 欄位 · 確認正確帶出物料的 External Material Group。

學習目標

- 能講出 BAdI 與 Classic User-Exit 的本質差異 (Interface/Class vs 寫死的 Function Module 編號) · 理解 BAdI 是 User-Exit 的 OO 化後繼者
- 能講出 Definition (SE18) / Implementation (SE19) 的分工 · 知道 Interface 為什麼一定要繼承 IF_BADI_INTERFACE
- 能分辨 Single Use 與 Multi Use 的差異 · 並各舉一個本課程已查證過的真實例子
- 理解 Filter-dependent 的用途 (依條件分流不同 Implementation)
- 能寫出正確的 GET BADI / CALL BADI / TRY...CATCH cx_badi_not_implemented 語法骨架
- 知道並非所有 BAdI 都能用新式關鍵字語法：沒有 ENHS/XS 身分的純舊式 BAdI 要用 cl_exithandler=>get_instance · 能判斷該用哪一種
- 理解「Implementation 存在」與「Implementation 生效 (Active)」是兩件事 · 能舉出本題查證的真實案例佐證；查一個 Multi Use BAdI 時知道要撈出全部同名前綴的 Implementation · 不能只看一筆就下結論

事前準備 (已於本系統 client 130 實際查證 · 非假設)

用 sap-adt MCP quickSearch 查 BADI_MM_* 系列 · 找到 BADI_MM_MATNR 這個真實標準 BAdI · GET 其 Enhancement Spot (/sap/bc/adt/enhancements/enhsxs/badi_mm_matnr) 確認：

```
<enhs:badiDefinition enhs:name="BADI_MM_MATNR" enhs:singleUse="false" ...>
  <enhs:interface adtcore:name="IF_EX_BADI_MM_MATNR"/>
</enhs:badiDefinition>
```

Interface IF_EX_BADI_MM_MATNR 讀出來含 interfaces IF_BADI_INTERFACE. (強制繼承的直接證據) + 13 個方法 (CHECK_MARA/CHECK_MARC/CHECK_MBEW/CHECK_MVKE/CHECK_MLGN/CHECK_MLGT/CHECK_MARD——物料主檔各張子表歸檔前的「使用中」檢查；ARCHIVE_MARA/ARCHIVE_MARC/...——歸檔本身的擴充邏輯；READ_ARCHIVE——從歸檔讀回資料) · GET 其 Enhancement Implementation (/sap/bc/adt/enhancements/enhoxh/badi_mm_matnr) 確認：

```
<enhs:badiImplementation enhs:name="BADI_MM_MATNR" enhs:isActive="false"
  enhs:runtimeBehaviorShorttext="The implementation will not be called">
  <enhs:implementingClass adtcore:name="CL_IM_BADI_MM_MATNR"/>
</enhs:badiImplementation>
```

CL_IM_BADI_MM_MATNR (套件 CINBADI) 讀出來可以看到真實方法本體 · 例如 ARCHIVE_MARC 呼叫 J_1IMTCHID_ARCHIVE_PUT / J_1IMODET_ARCHIVE_PUT (J_1I 字首是 SAP 印度在地化模組的命名慣例 · 證實這是印度稅務法規相關的物料主檔歸檔擴充) 。

題目需求

1. 完成三個真實 **BAdI** 案例對照表（可直接引用 Lecture 的表格，重點是理解每一格背後代表的意義）。
2. 解釋 **IF_BADI_INTERFACE** 這個「標記介面」的作用：如果一個 Class 直接 **INTERFACES if_ex_badi_mm_matnr**，但這個 Interface 沒有繼承 **IF_BADI_INTERFACE**，會發生什麼事？
3. 對照 **en02** 學過的 **Classic User-Exit** 啟用流程，說明 BAdI 的「Active 開關」跟 CMOD 的「Activate Project」有什麼異同（提示：都是「程式碼存在≠生效」的概念，但管轄範圍不同）。
4. 寫出呼叫 **BADI_MM_MATNR** 的 **check_mara** 方法的完整程式碼骨架，包含宣告、**GET BADI**、**CALL BADI**、例外處理。
5. 情境判斷：如果今天想在 SD 銷售流程裡，讓「大陸地區」跟「海外地區」的訂單分別走不同的客製化定價邏輯，但兩者共用大部分邏輯，應該設計成 **Multi Use** 搭配 **Filter**，還是乾脆寫兩個 **Single Use** BAdI？說明理由。

參考答案

三案例對照表：見 Lecture。

IF_BADI_INTERFACE 標記介面：如果沒有繼承這個標記介面，這個 Interface 就只是一個普通 OO Interface，SAP Kernel 的 BAdI 執行時期框架不會認得它、**GET BADI**/**CALL BADI** 這套機制根本不適用——這個 Interface 沒辦法被登記成 BAdI Definition，**SE18** 建立時就會擋下來要求繼承。

Active 開關 vs CMOD Activate Project：相同點——兩者都體現「程式碼寫好≠生效」；相異點——CMOD 的 Activate Project 是整個 **Project** 一次性的開關（一個 Project 可能同時掛好幾個 Enhancement Component），而 BAdI 的 Active 是掛在每一個 **Implementation** 自己身上的獨立屬性，不需要額外的 Project 概念，開/關單一 Implementation 不會影響其他 Implementation。

呼叫骨架：

```
DATA lo_badi TYPE REF TO if_ex_badi_mm_matnr.
DATA ls_mara TYPE mara.

TRY.
  GET BADI lo_badi.
  CALL BADI lo_badi->check_mara
    EXPORTING
      mdata = ls_mara
    EXCEPTIONS
      in_use = 1.
  IF sy-subrc <> 0.
    " 依 IN_USE 例外處理
  ENDIF.
CATCH cx_badi_not_implemented.
  " Multi Use，沒有 Active 的 Implementation 時通常直接略過，不一定會落到這裡
ENDTRY.
```

情境判斷：選 **Multi Use 搭配 Filter**（例如用「地區代碼」當 Filter 值）——兩地共用大部分邏輯代表核心程式碼應該只維護一份，Filter 機制讓系統依訂單的地區代碼自動選對 Implementation，之後如果又多一個地區，直接加一個新 Implementation + 對應 Filter 值即可，不需要動到既有程式碼；寫成兩個 **Single Use** BAdI 則完全做不到「共用大部分邏輯」這件事——**Single Use** 的設計前提就是全系統唯一，沒有「依條件切換到另一個 **Single Use** BAdI」這種機制。

思考題

1. **BADI_MM_MATNR** 底下印度稅務用的那個 Implementation (**CL_IM_BADI_MM_MATNR**) 目前是 Inactive，但 EAM / CRM S/4 用的另外 4 個是 Active——如果有一天使用者不小心手動把印度那個也 **Active** 了，物料主檔歸檔時會發生什麼事？這對正式環境的變更管理有什麼啟示？（提示：會開始真的呼叫印度稅務相關的歸檔擴充邏輯，若公司不是印度地區可能完全用不到甚至邏輯不合適；這說明 **Active / Inactive** 這種「一鍵開關」性質的設定，變更時要跟 **en02** 學到的傳輸請求 / CMOD Project 一樣審慎，不能隨手切換）
2. **en02** 的 **Classic User-Exit** (**EXIT_SAPLV01Z_001/002**) 沒有 **TRY...CATCH cx_badi_not_implemented** 這種保護機制，本題的 BAdI 有——這個差異除了「新舊技術有無這個例外類別」之外，還反映了什麼設計哲學上的差別？（提示：Classic User-Exit 假設「一定會被呼叫，只是預設是空的」；BAdI 假設「可能真的沒人實作」是一等公民狀態，需要程式主動處理「沒實作」這個分支，這也是為什麼 BAdI 比 Classic User-Exit 更適合設計成「可能永遠不會有人客製化」的擴充點）
3. **BADI_MM_MATNR** 的 Interface 有 13 個方法，橫跨 Check / Archive / Read 三種職責——如果要重新設計這個 BAdI，把它拆成三個各自獨立、職責單一的 BAdI (**BADI_MM_MATNR_CHECK** / **_ARCHIVE** / **_READ**) 會更好嗎？（提示：見仁見智，但可以討論：拆開的好處是職責清楚、可以分別開關；壞處是三個 Implementation 之間如果需要共享狀態或執行順序保證會變複雜——這也是為什麼很多 SAP 標準 BAdI 寧可讓一個 Interface 承擔多個相關職責，而不是拆到最細）

答案

主體不新建任何 SAP 物件，是觀念與真實案例查證：**BADI_MM_MATNR** (Enhancement Spot / Definition，**singleUse="false"**)、**IF_EX_BADI_MM_MATNR** (Interface，13 個方法，已確認繼承 **IF_BADI_INTERFACE**)、6 個 Implementation (4 Active / 2 Inactive，含 **CL_IM_BADI_MM_MATNR**) 均已用 **sap-adt** MCP 於 client 130 實際查證存在並讀出真實內容；並用驗證程式 **ZR_EN03_BADI_TEST** (**\$TMP**，快照 **zr_en03_badi_test.prog.abap**) 實測 **cl_exithandler=>get_instance** 對 **Multi Use** BAdI 一律成功回傳 bound instance，呼叫 **check_mara** 真的觸發了 **IN_USE** 例外 (**sy-subrc=1**)，證實有生效中的實作在跑。

延伸實戰（真實完成，見上方案例說明）：[CI_IOHEADER](#)（DDIC Structure・欄位 [ZZEXTWG](#)・快照 [ci_ioheader.tabl.abap](#)）、[ZCL_IM_WORKORDER_INFO](#)（Implementation Class・[TABLES_MODIFY_LAY](#) 方法・快照 [zcl_im_workorder_info.clas.abap](#)）均已建立並啟用；使用者在 [SE19](#) 完成 Implementation [ZWORKORDER_INFO](#) 的建立與 Activate，於 [COOIS](#) 實測確認新欄位正確帶出物料的 External Material Group。呼叫骨架、四案例對照表、情境判斷見本題內文。